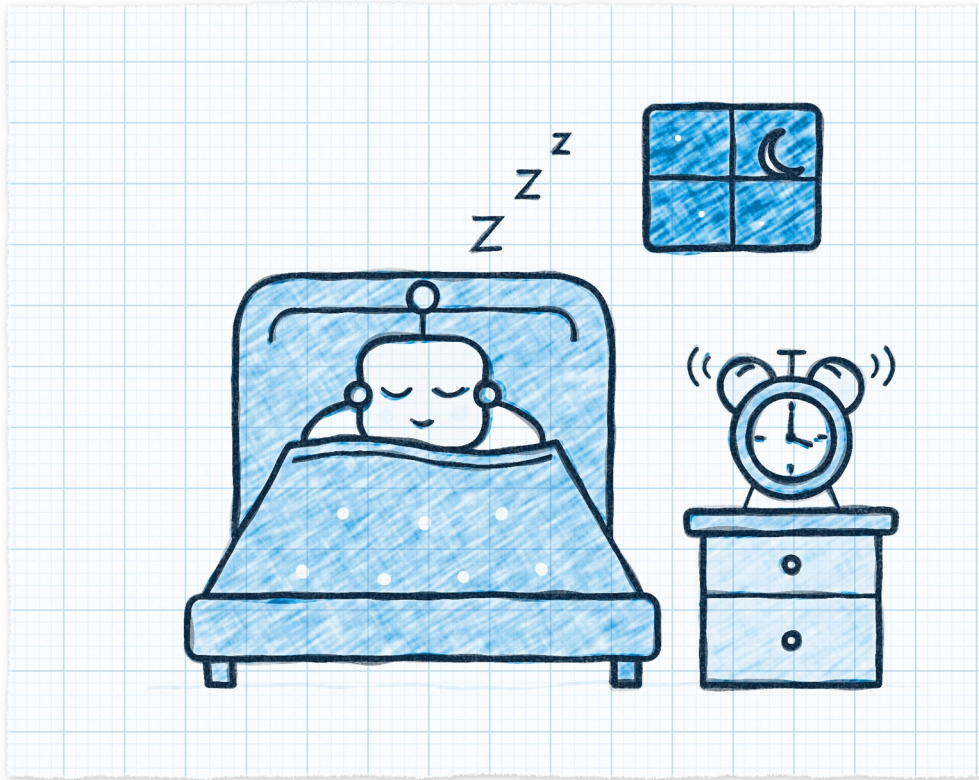




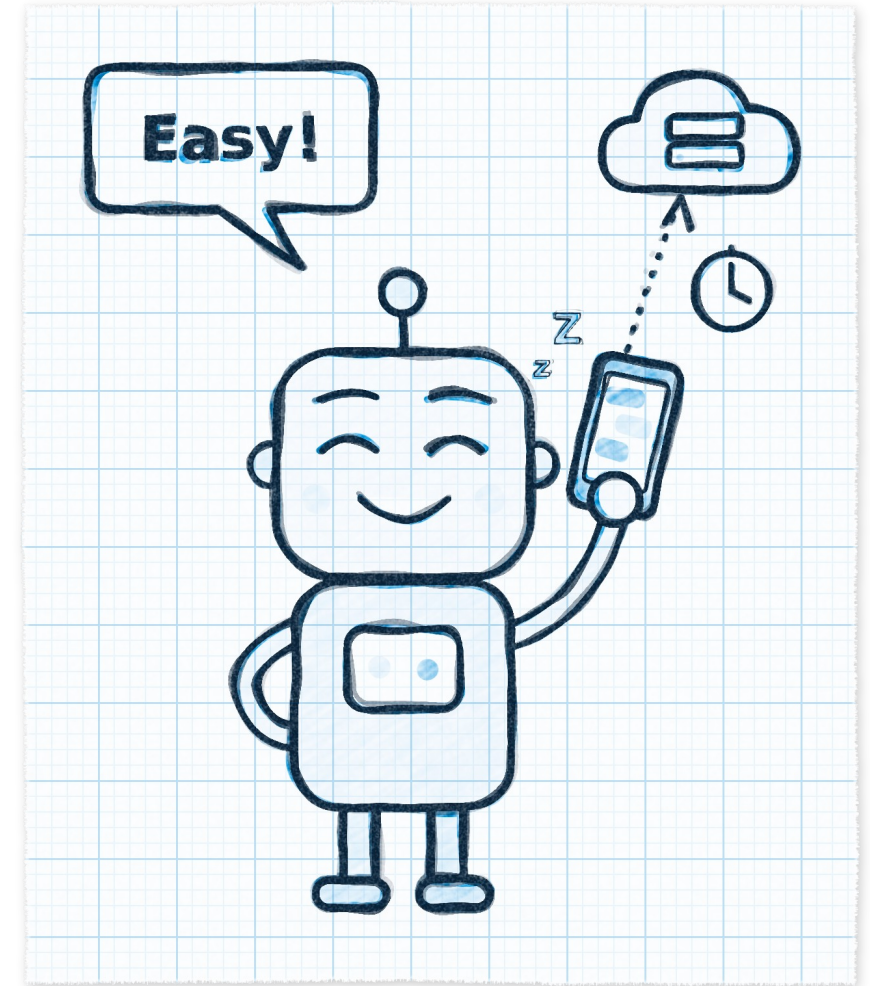
Waking Up Safely

Reducing Background Wake Locks with Android 17

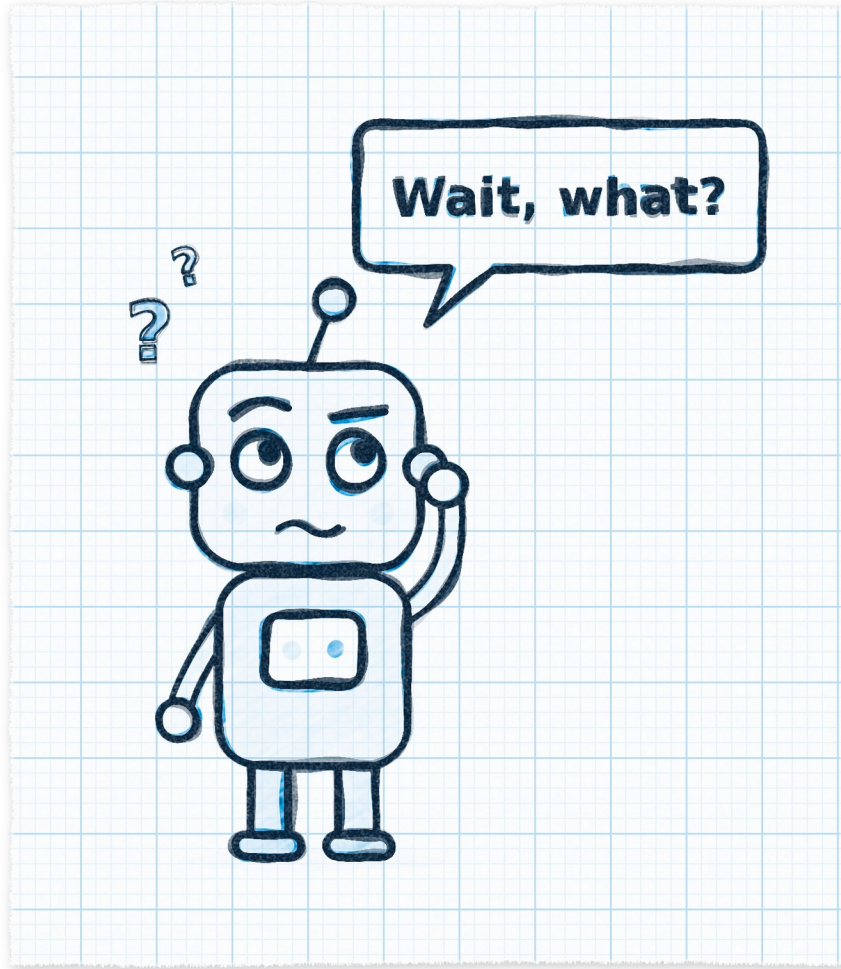
Thomas Künneth • Android GDE
droidCon Berlin, October 9, 2026

A seemingly small requirement

- A messaging app has to stay connected
- So every minute or so, it pings the server, even when the phone is idle
- We use the recipe everyone knows, with `AlarmManager`, a `PendingIntent` and a receiver
- It works, and nobody complains
- Months later, Android vitals shows wake locks, and not all of them come from code we wrote on purpose



What Android vitals tracks



- Excessive partial wake locks are a core vital, broken down by wake lock name
- A session counts as excessive at more than 2 hours of non-exempt wake locks in 24 hours
- Cross the 5% bad behavior threshold over 28 days, and Play may limit your app's visibility

Wake locks that system APIs take on your behalf count as yours. `AlarmManager`'s is named `*alarm*` (the asterisks are part of the name)

Three not so obvious questions

1. Where do those wake locks come from?
2. If we lighten the recipe, what happens when our process goes away?
3. How do we do that without leaving older phones behind?

Android 17 brings a new answer, a real improvement, and a trade-off
Curious? Let's start with ten years of "wake me up exactly then"

API	Year	What changed
19	2013	<code>setExact()</code>
23	2015	Doze, and <code>setExactAndAllowWhileIdle()</code>
24	2016	<code>OnAlarmListener</code> for <code>setExact()</code> , but not while idle
31	2021	<code>SCHEDULE_EXACT_ALARM</code> permission
34	2023	That permission denied by default for new installs targeting 33+

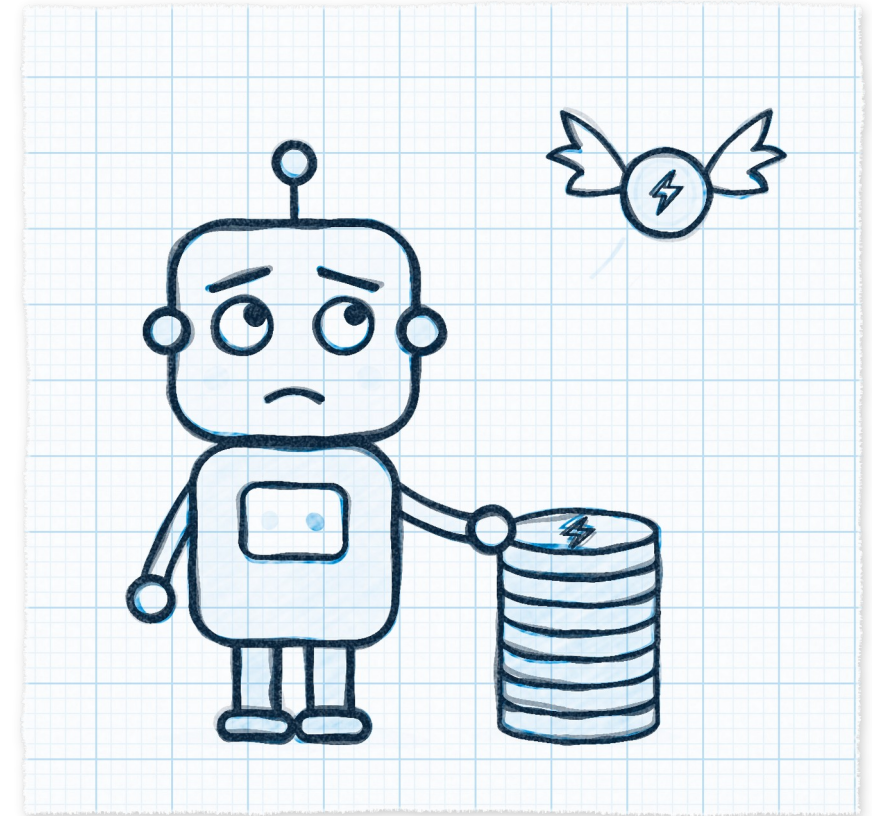
Look at API 23 and 24. Each gave us half of what we needed

- The listener (API 24) is light, but `setExact()` doesn't fire in Doze
- Firing in Doze (API 23) meant `setExactAndAllowWhileIdle()`, and that required a `PendingIntent`
- So if an alarm had to fire in Doze, it came with the whole broadcast machinery
- Our keep-alive has to fire while the phone sleeps, so it pays a tax for that machinery, every time
- Why not `WorkManager`? The Doze guide explains that it uses `JobScheduler` internally, so its tasks don't run in Doze

The tax on every fire

Every `PendingIntent` alarm triggers:

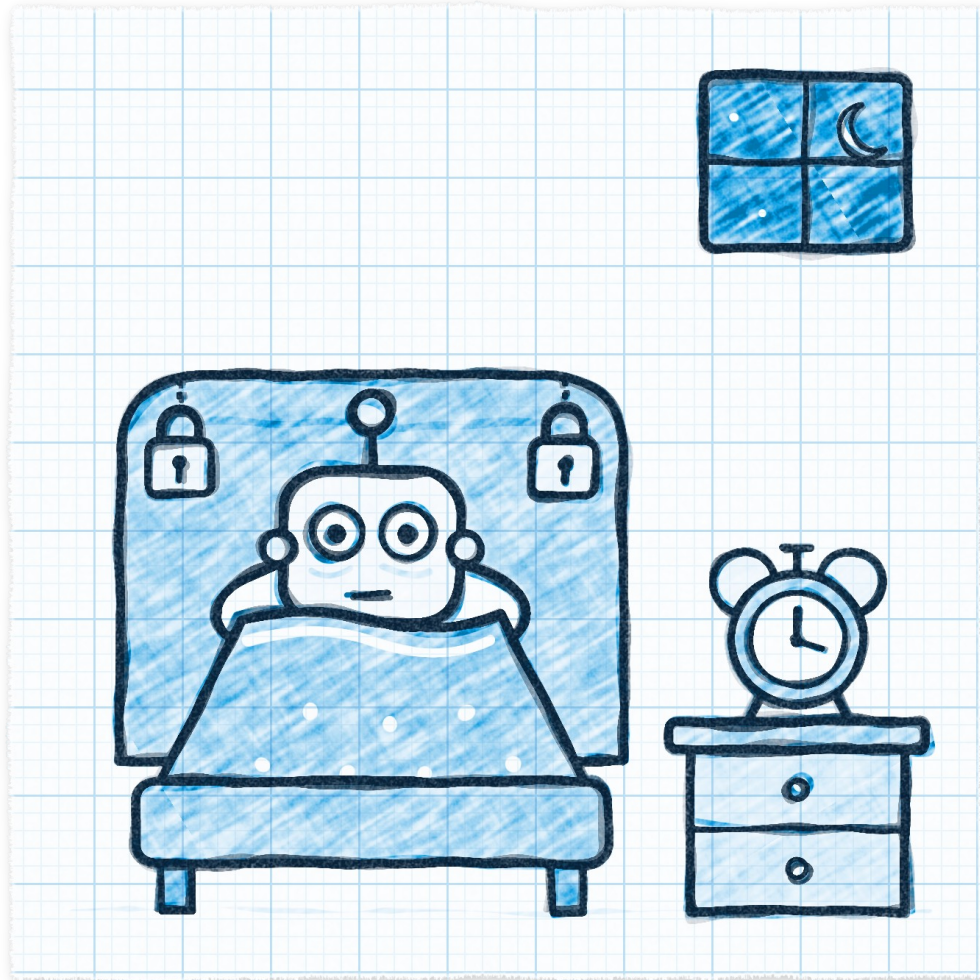
- An `Intent`, delivered across the process boundary
- A **cold start** if your process is gone
- `Application.onCreate()`, with every DI graph and SDK hanging off it
- Receiver instantiation, then `onReceive()`
- After dispatch, your app gets roughly **10 seconds** in which it may take **further** wake locks
- That window is an invitation to cascade



What the recipe makes us add in `onReceive()`

```
override fun onReceive(context: Context, intent: Intent) {  
    val pending = goAsync()  
    val wakeLock = powerManager.newWakeLock(  
        PowerManager.PARTIAL_WAKE_LOCK, "net:keepalive"  
    ).apply { acquire(30_000) }  
  
    scope.launch {  
        try { keepAlive.ping() }  
        finally {  
            if (wakeLock.isHeld) wakeLock.release()  
            pending.finish()  
        }  
    }  
}
```

- As the docs state, the alarm's wake lock is held while `onReceive()` runs, and released once it returns
- We hop to a coroutine, so we add `goAsync()`, and what the docs call "a separate wake lock policy"



- That makes two wake locks, the alarm's and our own. That's the cascade
- Forget to release ours, and the time adds up in your vitals

That answers **question 1**: the alarm's wake lock comes from the platform, our own from the recipe

Android 17 offers a lighter way

Allow-while-idle alarms can now be delivered to a listener. The Android 17 features page says it's

“ideal for apps that currently rely on continuous wakelocks to perform periodic tasks”

- Google names messaging apps that keep a socket connection alive as an example
- If FCM works for you, use it. The Doze guide strongly recommends it. The listener is for apps that need their own connection
- It helps apps that rely on a wake lock the whole time, and apps like ours that pay the tax on every alarm

While `setExact()` with a listener (API 24) takes tag, listener and `Handler`, the new `setExactAndAllowWhileIdle()` overload takes **tag, executor and listener**

```
alarmManager.setExactAndAllowWhileIdle(  
    AlarmManager.ELAPSED_REALTIME_WAKEUP,  
    SystemClock.elapsedRealtime() + 60_000,  
    "net:keepalive",    // tag  
    executor,           // java.util.concurrent.Executor  
    listener             // AlarmManager.OnAlarmListener  
)
```

The tag names your alarm in logs and **battery attribution**, so `net:keepalive` tells you which alarm it was

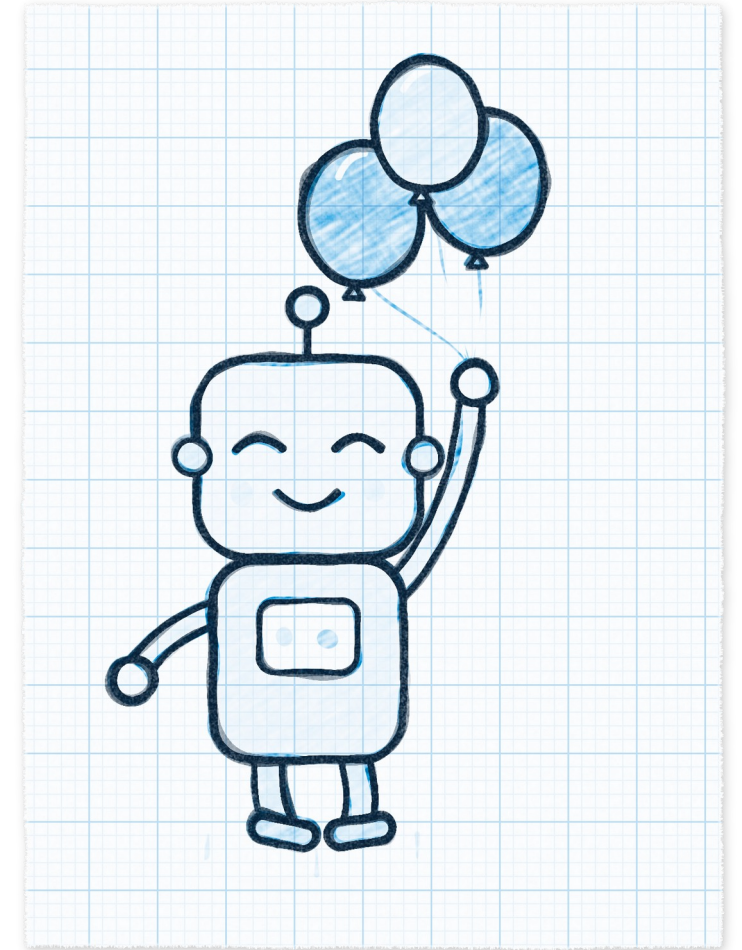
Same alarm, less tax

	PendingIntent	Listener (API 37)
Intent delivery across processes	every time	none
Cold start possible	yes, if the process is gone	no, the process must be alive
Receiver spin-up	every time	none
goAsync() + own wake lock	needed for async work	no receiver, no goAsync()

What that means

- When the alarm fires, the system calls an object that already exists, in a process that's already running
- As the Schedule alarms guide states, listener alarms don't need `SCHEDULE_EXACT_ALARM`¹. The `PendingIntent` path does when targeting 31+, with documented exceptions
- Google gives no savings figure. Your vitals will show how much you save
- There's a catch. The process must be alive. More on that shortly

¹ Not on Android 12 and 13 (S, [TIRAMISU](#)): apps targeting 31+ needed it for listener-based `setExact()` too, unless exempt from battery restrictions



Our keep-alive on API 37

```
@RequiresApi(Build.VERSION_CODES.CINNAMON_BUN)
class Heartbeat(
    private val alarmManager: AlarmManager,
    private val executor: Executor,    // single thread
    private val socket: KeepAliveSocket,
) : AlarmManager.OnAlarmListener {
    private var running = false    // only touched on executor

    fun start() = executor.execute {
        running = true; scheduleNext()
    }
    fun stop() = executor.execute {
        running = false; alarmManager.cancel(this)
    }

    private fun scheduleNext() =
        alarmManager.setExactAndAllowWhileIdle(
            AlarmManager.ELAPSED_REALTIME_WAKEUP,
            SystemClock.elapsedRealtime() + INTERVAL_MS,
            "net:keepalive", executor, this,
        )
    // onAlarm() follows on the next slide
}
```

- **Heartbeat** is the listener itself. **start()** schedules the first alarm, and every alarm schedules the next
- **stop()** passes **this**, because cancelling only works with the same object you scheduled
- Everything runs on one background thread, because the ping is network I/O and must stay off the main thread

Here's `onAlarm()`, which runs every time the heartbeat fires

```
override fun onAlarm() {  
    if (!running) return           // late delivery  
    scheduleNext()                 // first: one-shot  
    runCatching { socket.ping() } // keep looping  
}
```

- The flag catches a delivery already on its way when `stop()` ran
- One-shot, so the listener reschedules itself first, before anything can fail
- A failed ping must not end the loop. An exception in `onAlarm()` would crash the app
- The docs don't promise a wake lock around `onAlarm()`, so keep the work short and don't hand it off

What about Doze?

"Allow while idle" doesn't mean "whenever you like"

- For the **PendingIntent** overload, the docs allow about **once a minute** at most, in normal operation
- In deep idle, Google's own pages disagree. The reference says "such as 15 minutes", the Doze guide once per nine minutes, the power limits page 7 per hour
- The system may **reorder** these alarms, even against your own app's alarms
- So our "every minute" from the story may not hold in deep idle, or in a lower standby bucket
- The new overload's docs say nothing about throttling, and nothing about being exempt. So plan as if it isn't

The catch with listener alarms

- As the new overload's own docs warn, listener alarms may be **canceled** once your process has **no components running**, such as an [Activity](#), a [Service](#) or a [ContentProvider](#)
- The [OnAlarmListener](#) docs add that the requester must keep running until delivery, "or delivery will fail"
- That's broader than "the process got killed"
- A [PendingIntent](#) lives in the system; a listener lives **in your heap**

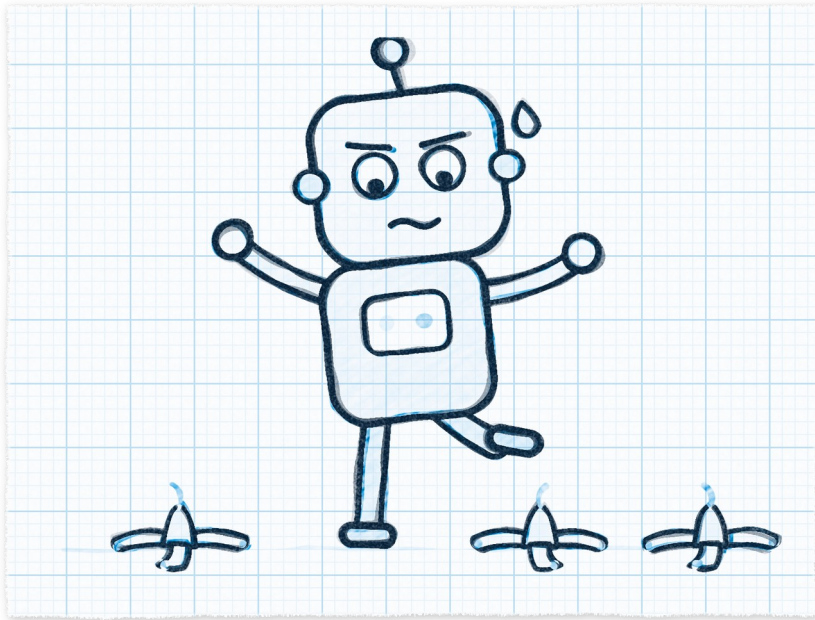
Which alarm to choose?

Question	Mechanism
Must it fire even if my app is dead?	PendingIntent , on every API level
Only while something of mine runs?	Listener , on API 37+

- A reminder due in a few hours → **PendingIntent**, even on API 37
- A socket heartbeat → listener. If the process dies, the socket is dead too
- In our story, the **PendingIntent** cold-started a process whose connection had died with it

That answers **question 2**: the lighter alarm goes away with the process, so decide by delivery need

Three pitfalls



- **Cancel** when your component finishes, as the reference asks. Pass the listener you scheduled, because a new lambda is a different object and would cancel nothing
- **Process death** ends the listener, so let a service that owns the socket own the heartbeat. Schedule in `onCreate()`, cancel in `onDestroy()`, so every new start schedules again
- **Throttling** delivers pings late and out of order, within the documented limits. A server timeout longer than the worst gap avoids reconnects, which is the cheapest battery fix in this talk

One call site, two mechanisms

```
enum class Delivery { WHILE_RUNNING, ALWAYS }

interface AlarmDispatcher {
    fun schedule(tag: String, at: Long, policy: Delivery, work: () -> Unit)
    fun cancel(tag: String)
}
```

Callers say what they need, the dispatcher picks the path

- **ALWAYS** means it must survive our process, so it goes through **PendingIntent**, everywhere
- **WHILE_RUNNING** is the only place the new API applies
- Cancel **by tag**, so you don't have to hold on to the same object. The dispatcher does that for you

The fork, in one place

```
fun alarmDispatcher(context: Context): AlarmDispatcher =  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.CINNAMON_BUN)  
        HybridDispatcher(  
            ListenerDispatcher(context),  
            BroadcastDispatcher(context),  
        )  
    else  
        BroadcastDispatcher(context)
```

- On API 37+, [HybridDispatcher](#) picks the path by policy. Below 37, everything goes through broadcasts
- The API level check happens here; callers never see it

Routing by policy

```
class HybridDispatcher(  
    private val listenerPath: AlarmDispatcher,  
    private val broadcastPath: AlarmDispatcher,  
    ) : AlarmDispatcher {  
    override fun schedule(  
        tag: String, at: Long,  
        policy: Delivery, work: () -> Unit,  
    ) {  
        cancel(tag)    // the tag may sit on the other path  
        when (policy) {  
            Delivery.WHILE_RUNNING ->  
                listenerPath.schedule(tag, at, policy, work)  
            Delivery.ALWAYS ->  
                broadcastPath.schedule(tag, at, policy, work)  
        }  
    }  
  
    override fun cancel(tag: String) {  
        listenerPath.cancel(tag)  
        broadcastPath.cancel(tag)  
    }  
}
```

- Which alarm to choose now lives in code, where it can be reviewed
- Behind `listenerPath`, `ListenerDispatcher` is our heartbeat, keyed by tag. One listener per tag, so cancel by tag finds it, and rescheduling replaces it
- Create the dispatcher once and share it, for example through DI

The broadcast side

`BroadcastDispatcher` is our recipe from the start, now behind the interface

- For `ALWAYS`, the lambda only covers the running process. After a cold start, work comes from a **registry**, filled by tag at app start. API 37 removes that cost for `WHILE_RUNNING`
- Check `canScheduleExactAlarms()` on API 31+, and have a plan for when it's revoked
- Async work in `onReceive()` needs its own wake lock, because the docs release the alarm's lock on return. Keep it short, release it in `finally`

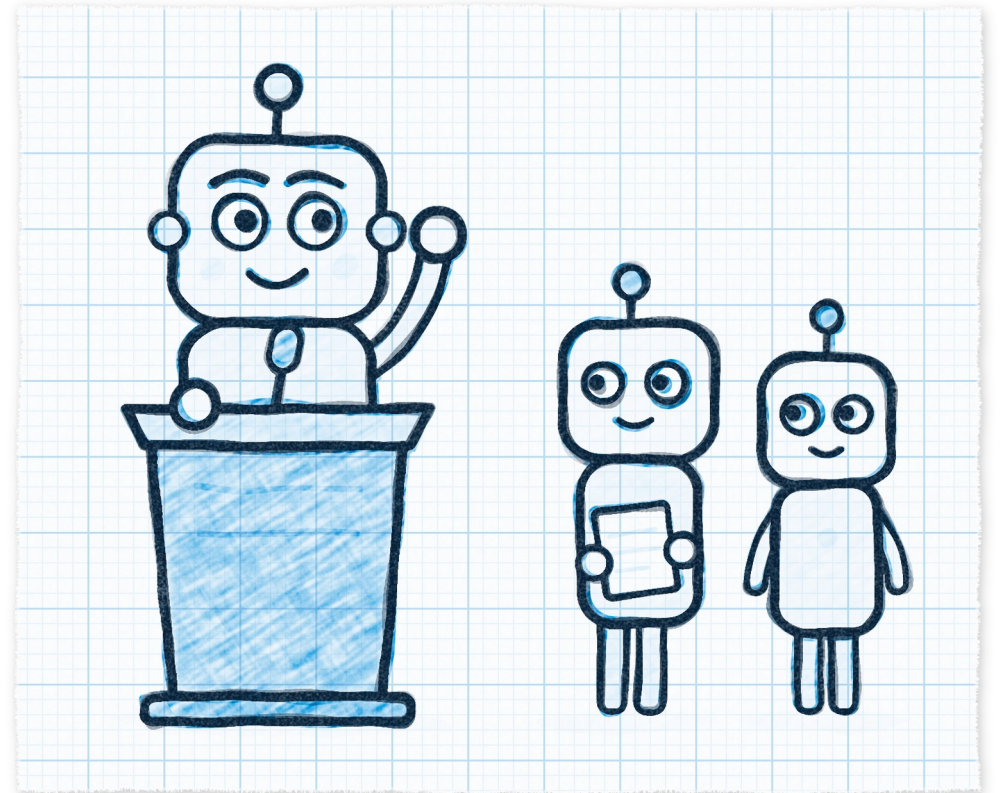
Both branches of the fork

Test **both branches of the fork**, on API 36 and API 37 emulators

That answers **question 3**: older phones keep the path they always had, and API 37 adds a lighter one

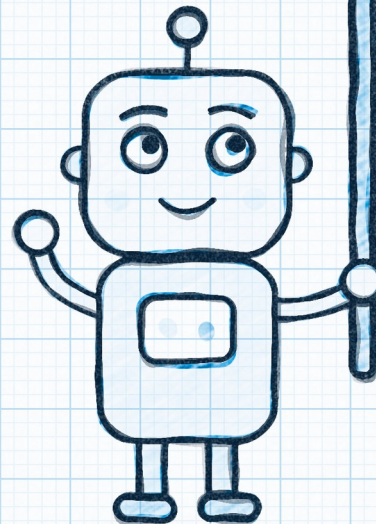
What I'll tell my team

- **Ask first** whether this alarm needs to survive our process
- **Fork once**, behind an interface, never at the call site
- **Own and cancel**, because listener alarms live and die with your components
- **Tag everything, measure in idle**, then go back to that vitals screen
- The new API isn't magic. It's the platform no longer making us pay for components we never needed





<https://github.com/tkuenneth/WakingUpSafely>



Thank you

Questions now, or any time later:

GitHub	<u>tkuenneth</u>
LinkedIn	<u>tkuenneth</u>
Mastodon	<u>@tkuenneth@mastodon.social</u>
Bluesky	<u>@tkuenneth.dev</u>
Web	<u>tkuenneth.dev</u>
Email	<u>mail@tkuenneth.dev</u>

Developer. Author. Speaker. Mentor.



References

- [Excessive partial wake locks metric is out of beta — blog post](#) · [Identify and optimize wake lock use cases](#)
- [Android 17 features: listener support for allow-while-idle alarms](#)
- [AlarmManager reference](#) · [The API 37 overload](#)
- [OnAlarmListener reference](#)
- [Schedule alarms — exact alarm permission, listener exception](#)
- [Android 14: exact alarms denied by default](#)

References, continued

- [Optimize for Doze and App Standby](#) · [Power management resource limits](#)
- [Broadcasts overview — manifest-declared receivers](#)
- [Application reference — onCreate\(\)](#)
- [SDK Platform release notes — API levels and release dates](#)